

The Alberta Buck - Basket Operations (v1.0)

One Filter Bank, Two Mandates: Commodity Rebalancing and Monetary Operations

Perry Kundert

2026-08-16



The BuckBasket rebalances commodities against each other. It reads a seven-scale filter bank over its pool prices, finds which constituent is rich and which is poor, and moves value between them as excursions turn. The `pairs` engine works in *differential mode* and cancels the common mode deliberately, because "BUCK's own valuation is the K-controller's job".

But a Uniswap tick is a log price, so the MEAN of those legs is BUCK's own valuation – `basketValueInBuck`, the controller's process variable. The signal is already in the filter bank. The basket computes it every cycle and discards it.

This article puts it to work. The same ladder drives two mandates: the differences steer commodity rebalancing, the mean steers *monetary operations*. The basket becomes a market maker in BUCK whose quote skew is set by the controller's error, and whose inventory changes *are* the open market operation.

The purpose is speed. K works, but it works slowly: it acts only through `creditLimit`, and a credit book turns over in months. An attacker with real money can push BUCK off parity faster than K can pull it back. A basket that trades the common mode acts in hours, absorbs the excursion while it is forming, and books the attacker's loss as depositor profit.

In simulation, over nine seeds, an inflation attack's peak excursion falls from 1538bp to 1099bp, the basket books \$604k, and the attacker is worse off in nine runs out of nine. The same machinery LOSES when the move is real rather than an attack, which is the property that makes the rest of it trustworthy.

Getting there required three separate bounds, and each was invisible until the one before it was in place. Without them the basket accumulated half the pool, showed a profit the whole way, and never once raised an alarm – because the act of absorbing suppresses the very signal that would say stop. That sequence is documented here in full, because it is more useful than the result.

Contents

1	The signal is already there	3
2	Why speed matters	3
3	The four quadrants	3
3.1	Q1 and Q3 are repo	4
3.2	Q2 and Q4 are outright	4
4	What produces what	4
5	What it measures	5
6	The constraints, and one of them is a correctness bound	5
6.1	Measure fast enough, or you become the destabilizer	5
6.2	Depositor claims must not be diluted	7
6.3	Reserves are finite and the bound is public	7
6.4	Anti-windup	7
7	Three runaways, each hiding behind the fix for the last	7
7.1	One: the operator suppresses its own alarm	7
7.2	Two: the fix relocated the runaway	8
7.3	Three: only then does the failure test pass honestly	8
8	Implementation	8
8.1	The director gains a second output	8
8.2	The basket gains one keeper entry point	8
8.3	Implementing the agent against the chain simulation	9
8.4	Order of work	12
8.5	What the agent showed, including the part that did not survive	12
9	The contract: BuckBasketOps	13
9.1	The depositor separation is achieved by NOT writing to the payout path	13
9.2	What only the contract can do	13
9.3	A bound that is not politeness	14
9.4	The desk has to be capitalized, and that is the 1992 lesson	14
9.5	What the contract desk did	14
9.6	Why the limits exist, and what they are actually for	15
10	Limitations	16

1 The signal is already there

The **pairs** rebalance engine keeps, for each constituent, a ladder of seven EMAs over the pool tick. A tick is log of the constituent's price *in BUCK*. Write c_i for leg i 's log price. Then

$$\underbrace{c_i - \bar{c}}_{\text{differential}} \quad \text{and} \quad \underbrace{\bar{c} = \frac{1}{N} \sum_i c_i}_{\text{common}}$$

are the two halves of the same reading. The differences say which commodity is rich against which. The mean says what the whole basket costs in BUCK – which is **basketValueInBuck**, the number the PID steers.

The EMA is linear, so this costs nothing:

$$\frac{1}{N} \sum_i \text{EMA}_k(c_i) = \text{EMA}_k\left(\frac{1}{N} \sum_i c_i\right)$$

The common mode at every one of the seven scales is the mean of the per-leg ladders the engine already maintains. No new state, no new reads, no new storage. The basket has been computing BUCK's own mispricing at seven timescales and throwing it away every cycle.

2 Why speed matters

BUCK_K is a sound controller and a slow one. It acts through **creditLimit**, so its only lever is how much new credit-backed BUCK may be issued. A credit book turns over in months. When someone mints a fortune against pledged wealth and spends it, K responds by tightening the tap – and the tap governs a flow that takes quarters to matter.

Meanwhile the attack lands in days.

The BuckBasket is the largest pool of real assets in the system and it can act immediately. It already holds both sides of every pool. Giving it the common-mode signal turns the system's inertia into its fastest actuator:

- **K sets the standing policy.** Slow, structural, and it decides how much BUCK the economy may carry.
- **The basket runs the operations.** Fast, bounded, reversible, and it decides what happens this week.

That is the division a central bank already makes between its policy rate and its open market desk.

3 The four quadrants

Two questions decide the operation. Is BUCK cheap or dear? And has the excursion turned, or is it persisting?

BUCK CHEAP ($\text{bvib} > 1$)	BUCK DEAR ($\text{bvib} < 1$)
basket costs more BUCK	basket costs less BUCK
than it should	than it should

REVERTING the excursion has turned	Q1	ABSORB (temporary) TOKEN-funded bids BELOW the price. Buy the dump; sell it back on recovery. Net supply: unchanged.	Q3	SUPPLY (temporary) BUCK-funded offers ABOVE the price. Sell into the bid; buy back on the return. Net supply: unchanged.
PERSISTENT past the leash for N days	Q2	RETIRE (outright) Burn acquired BUCK against outstanding. Supply FALLS.	Q4	ISSUE (outright) Mint against basket TOKEN and sell. Supply RISES.

3.1 Q1 and Q3 are repo

A concentrated Uniswap V3 range order performs an operation and unwinds it automatically. Place BUCK above the price: as BUCK strengthens the position sells it, and as BUCK returns the position buys it back. The operation reverses itself, and it earns the spread while it waits.

That is what a temporary open market operation is. A central bank uses repo for daily noise precisely because it self-reverses and needs no decision to unwind. The basket gets the same instrument for free from the AMM it already lives in.

Temporary operations are the default. They are cheap to be wrong with.

3.2 Q2 and Q4 are outright

An outright operation changes the size of the balance sheet. Burning BUCK retires it permanently; minting creates it permanently. These are reached only when the temporary operation has been consumed *and* the price has stayed away.

Persistence is measured as a duration, not a velocity. This matters: a short-window velocity changes sign on noise, so a "has it turned" test built on velocity almost never reports "no". In the first version of the model the outright quadrants were unreachable for exactly this reason – supply never moved, and half the mechanism was inert while looking healthy.

The rule is: the deviation exceeds the leash on every one of N consecutive days. Thirty days at 200bp in the model.

4 What produces what

Ordinary commodity dispersion. cbBTC rises against LABR; the basket's weights drift apart; the mean does not move. This is *differential* and it goes to commodity rebalancing exactly as today. No monetary operation, because BUCK is not mispriced – the commodities have moved relative to each other.

A whale mints and spends a fortune. Every pool's BUCK reserve rises together. The differences barely move – the whale sold BUCK, not one commodity – so the rebalancer sees almost nothing. The MEAN moves hard. This is *common* and it is Q1: the basket bids for BUCK below the market with TOKEN it already holds, and buys the dump at a discount.

The whale's position persists. The excursion does not turn. After thirty days past the leash the basket escalates to Q2, burning the BUCK it has accumulated against `totalOutstandingBuck`. Supply falls permanently. The whale must eventually buy BUCK back to retire the credit line that funded the attack – into a market whose float the basket has removed.

Someone wants BUCK badly and bids it up. The mean moves the other way. Q3: the basket offers BUCK above the market from inventory. If the bid persists, Q4 mints against basket TOKEN and sells into it, acquiring real assets with money the market has over-valued.

A genuine revaluation. Commodities really do become worth more BUCK, and stay there. This looks exactly like an attack to the signal, and it is the case the design must lose gracefully. See *When it should lose* below.

5 What it measures

The model is `alberta_buck/sim/monetary_ops.py`. It abstracts the pools to a single price with constant-product impact against depth, mean-reverting at a 120-day timescale, and runs an attacker who sells 6M BUCK over 20 days, holds 60 days, then unwinds – because an inflation attack is funded by drawing credit, and the credit must be retired by buying BUCK back. Costs are 30bp a leg. Operations are bounded at 0.4% of depth per day, outright at 0.2%.

Means over nine seeds, because a single draw proves nothing – the first version of this table was built on one seed and reported the opposite conclusion in the `drift` row:

scenario	peak dev off	peak dev ON	days off 2%	basket P&L	profitable
inflate	1538bp	1099bp	244	+603,772	9/9
squeeze	1366bp	1274bp	208	+491,509	9/9
drift	1141bp	870bp	313	-96,234	2/9
quiet	760bp	509bp	135	+147,392	9/9

The excursion is damped in every scenario, the basket is paid for absorbing it in the three where the move is not real, and it **LOSES** where the move is real. Against the attacker, over nine seeds, operations cost the attacker a further \$256k on the inflation attack and \$295k on the squeeze – worse for the attacker in 9 runs out of 9 in both cases.

6 The constraints, and one of them is a correctness bound

6.1 Measure fast enough, or you become the destabilizer

`basketValueInBuck` is read from the basket’s own pools, so a basket that trades on it moves its own sensor. The obvious guard is to measure on a scale so slow that one bounded operation cannot shift it. That was the first design, measuring on the 160-day EMA.

It made things worse. The excursion peaks at 1645bp on spot but only 712bp on the 160-day average, and that 933bp of attenuation is phase lag. Operations timed off a lagged signal arrive after the spot has turned, so their flow lands pro-cyclically. Peak deviation grew from 1645bp to 2116bp.

They still made money. That is the uncomfortable part, and the central result of this model:

Profitability and stabilization are not the same objective. "The corrective trade is the profitable trade" holds only when the signal is timely. With lag they come apart, and a lagged operator is a profitable destabilizer.

Measured across the ladder, on the inflation scenario:

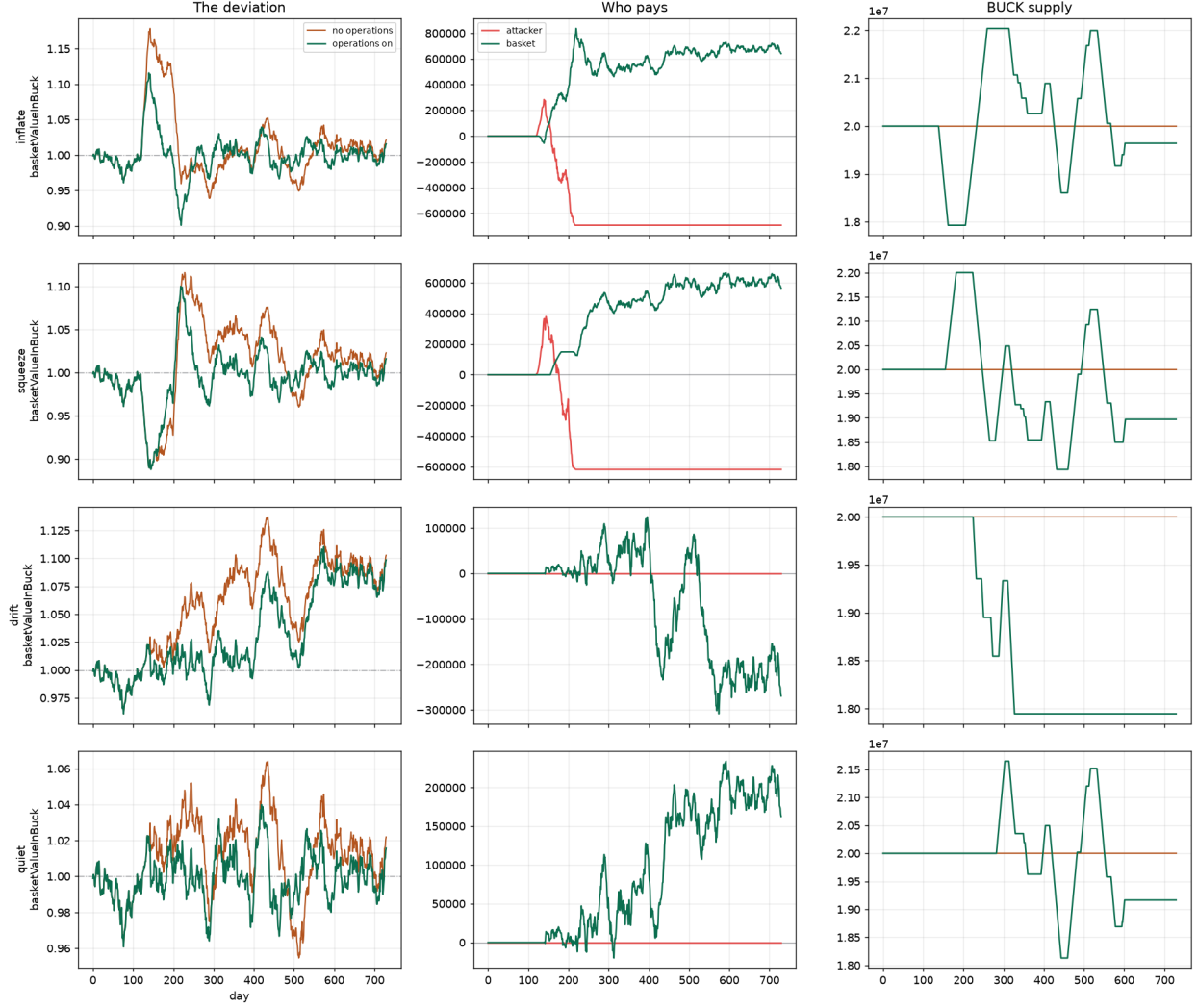


Figure 1: Four scenarios, with operations off and on. Left: the deviation the market actually experiences. Middle: who pays – the attacker’s mark against the basket’s. Right: BUCK supply, which only the outright quadrants move. The bottom row is the honest failure case.

measure on	peak dev	damping	basket P&L	attacker P&L	supply change
10d	1004bp	-641bp	+623,186	-580,548	-566,628
20d	1074bp	-570bp	+672,384	-622,660	-801,908
40d	1185bp	-460bp	+694,414	-711,547	-623,610
80d	1344bp	-301bp	+746,063	-985,063	+685,152

Two things fall out.

Stabilization and extraction trade off. Acting early damps the excursion, and therefore forecloses the profit that excursion would have paid. The basket cannot both prevent the attack and maximally loot it. This is a policy choice, and it should be made explicitly rather than discovered.

Past roughly 80 days the outright operations invert. The lagged reading still says "dear" long after the market has gone cheap, so an inflation attack is answered by ISSUING more. That is not a preference, it is a correctness bound on the measurement scale.

20d is the knee. Collocation at that scale is handled by the size bound instead: an operation capped at 0.4% of depth per day cannot dominate a 20-day average.

6.2 Depositor claims must not be diluted

A depositor's claim is $\text{redeemBuck} \times \text{NAV} / \text{totalOutstandingBuck}$. BUCK minted for a monetary operation is not backed by a depositor's TOKEN, so folding it into the same counter would dilute every depositor.

Monetary issuance must be tracked as its own liability class. Depositor claims are computed against NAV net of the monetary book.

6.3 Reserves are finite and the bound is public

A convertibility undertaking is only as good as its reserves. The basket's defensive capacity is its NAV, and any bound it publishes will be probed – the 1992 ERM crisis is the standing lesson. Size operations against NAV, and expect the bound to be tested rather than respected.

6.4 Anti-windup

Bounding the operation while K keeps demanding more winds the PID integrator against the bound. Feed the *achieved* action back to the integrator, not the requested one.

7 Three runaways, each hiding behind the fix for the last

The **drift** scenario – a genuine 8% revaluation, no attacker – is the test that matters, because a design which profits in every scenario is not being tested. Building it exposed three separate unbounded behaviours, and each was invisible until the previous one was fixed. This section is the most useful thing in this article.

7.1 One: the operator suppresses its own alarm

Absorbing holds the measured deviation down. That is what absorbing IS. So a persistence test built on that deviation is silenced by the very act it exists to police.

In the drift scenario the basket bought until its BUCK inventory reached **49.9% of pool depth, and still climbing**, having spent essentially its entire TOKEN reserve – and the outright quadrants never once fired, because its own bid held the price inside the leash the whole time.

Worse, it showed a PROFIT throughout, in 7 of 9 seeds. The gain was real volatility harvesting around the prevailing level; the loss was an unrealized mark on a position that could not have been unwound. A P&L that looks healthy while the position becomes the market is the classic shape of a blow-up, and nothing in the price signal reported it.

The fix is to escalate on INVENTORY rather than on price. Inventory is the one signal the operator's own action cannot suppress, because it *is* the operator's own action. If a large position has been absorbed and the price has not come back, the move is real – whatever the price says.

7.2 Two: the fix relocated the runaway

With inventory-based escalation the position limit held – 49.9% of depth fell to 5.3% – and the basket promptly burned **18.65M BUCK, 93% of all supply**, chasing a gap that 8% would have closed. Escalation fired every day and nothing bounded the cumulative balance-sheet change.

The fix is a cumulative bound on outright operations, 10% of supply here. A central bank announces the size of a programme; it does not run the desk until the number comes right.

7.3 Three: only then does the failure test pass honestly

With all three bounds – per-operation size, position limit with inventory-based escalation, and cumulative balance-sheet cap – the drift scenario loses in 7 seeds of 9, and the supply change lands at 1.99M against a 2.0M cap for an 8% revaluation. The right magnitude, in the right direction, and a loss.

The general lesson is worth stating on its own. Each bound was necessary, each was invisible until the previous one was in place, and the system looked healthy and profitable in between. A bounded operator needs bounds on the FLOW, on the POSITION, and on the CUMULATIVE BALANCE SHEET, and they are three different constraints rather than three settings of one.

8 Implementation

8.1 The director gains a second output

RebalanceDirectorBase already caches per-leg ladders and folds them into per-leg efforts. The common mode is the mean of those ladders, so:

```
/// Signed monetary effort, bp of NAV per epoch.  
/// > 0 issue BUCK < 0 retire BUCK  
/// 'outright' distinguishes a balance-sheet change from a range order.  
function monetaryEffort() external view returns (int32 effortBp, bool outright);
```

The computation is a mean over the cached `m[k]` the engine already holds, a deadband, a leash, and a consecutive-day counter. No new per-leg state.

8.2 The basket gains one keeper entry point

`monetaryOperation()`, permissionless and bounded, mirroring the existing `rebalanceSource` / `rebalanceSink` pattern:

1. read `monetaryEffort()`;

2. if not `outright`, place or refresh a concentrated range order on the indicated side, sized by the effort and capped by the per-epoch bound;
3. if `outright`, burn from the monetary book (retire) or mint into it (issue), within the same bound;
4. account to `monetaryOutstanding`, never to `totalOutstandingBuck`.

8.3 Implementing the agent against the chain simulation

Step 2 below is the one to do first, and this section is what it needs. Every item here cost a debugging cycle while building `BuckPoolInvestorAgent` and `BuckIssuerArbAgent`; none of it is visible from the contracts alone.

8.3.1 Where it goes

A `_ProxyAgent` subclass in `alberta_buck/sim/equilibrium_agents.py`, registered with `@register` and added to the `REBALANCING` roster. The base class provides `_bind_proxy`, `_proxy_exec`, `_buy_buck`, `_sell_capped` and `_swap_via_simlp`. The proxy is a `SimLP`, which matters: it can serve the Uniswap V3 mint callback itself, so the agent can hold concentrated positions directly.

8.3.2 Reading the signal

`basketValueInBuck()` off the basket is the spot common mode. The agent keeps its own EMA ladder over `log(bvib)` on the `PairsRebalanceDirector` window geometry (5,10,20,40,80,160,320), measures the level on the 20-day rung, and detects the turn on the 10-day rung. Advance the ladder ONCE per day, before any decision, or the level lags the action by a tick.

8.3.3 The three primitives, and their exact call paths

Range order. The proxy mints directly:

```
d.chain.send(self.proxy.functions.mint(pool, tickLower, tickUpper, L, t0, t1))
```

Exiting takes TWO calls through `exec`, and a burn alone silently strands the capital – `burn` only credits the owed amounts, `collect` moves them:

```
self._proxy_exec(d, pool, pool_c.encode_abi(
    "burn(int24,int24,uint128)", args=[lo, hi, liquidity]))
self._proxy_exec(d, pool, pool_c.encode_abi(
    "collect(address,int24,int24,uint128,uint128)",
    args=[self.proxy.address, lo, hi, 2**128 - 1, 2**128 - 1]))
```

Issue. `Buck.mint(amount)` ACTIVATES coverage against the agent's `BuckCredit`; it does not credit the balance. Spending afterwards is what draws. Two consequences that look like bugs if they are not expected: `BuckCredit.currentValue` returns 0 while `activatedValue` is zero, so an un-activated NFT contributes nothing to `creditLimit` and the agent can hold collateral it cannot spend; and remaining capacity must be read from chain, never tallied in Python, because depreciation moves it underneath any local count. Note the two halves live on DIFFERENT contracts:

```

face = d.credit.functions.depreciatedFaceValue(tid).call()    # BuckCredit
used = d.buck.functions.mintsBacked(tid).call()              # Buck
headroom = max(0, face - used)

```

A Python-side tally of what was activated drifts from the chain within days and presents as "insufficient credit allocation" on most attempts.

Retire. Buy BUCK back so the signed balance climbs toward zero, then burn to release coverage. `Buck.burn(uint256)` or the tokenIds overload.

8.3.4 An agent is not the basket, and two quadrants are weaker for it

This is the one place where the quadrant table above does not survive contact with the contracts, and it is worth being exact because the difference is what phases 3 and 4 exist to close.

`mintFromBasket` and `burnFromBasket` both begin

```

require(msg.sender == basket && basket != address(0), "BUCK: not basket");

```

so an agent reaches supply only through the ordinary credit path. Three consequences, none of them obvious from the quadrant table:

Q4 is bounded by K; the basket's is not. An agent mints against its own `BuckCredit`, so it is capped by `creditLimit = totalCurrentValue x buckK.mintFromBasket` bypasses that machinery entirely – direct-mint BUCK is backed by the basket's TOKEN reserves rather than by insured-asset credit. The funding factor is NOT the constraint here: it is $1 + 10(b - p)/b$, which returns ZERO once BUCK is about 10% over-valued, so the gate is switched off exactly when Q4 wants to fire.

Burning does not destroy tokens. `Buck.burn` deactivates coverage – "you repay the credit, then release the coverage, as with any loan" – and the post-burn check is `used < creditLimit`. Nothing is subtracted from the caller's balance.

So retirement happens on the BUY, not on the burn. The supply invariant is

$$\text{totalSupply} = \sum_a \max(0, \text{signedRaw}(a))$$

An account holding drawn credit has a negative signed balance and therefore contributes nothing. Buying BUCK while drawn moves float out of the pool and into an account where it counts for nothing, and supply falls by exactly that amount. Buying BEYOND the draw just accumulates inventory and changes no supply at all. The burn's job is to close the line so the retirement cannot simply be undone by redrawing – which is precisely the temporary/outright distinction, and it is the same distinction Q1 and Q2 encode.

Two things follow for the agent.

It needs a STANDING BOOK. With no drawn line, Q2 has nothing to retire and degrades to Q1 whatever the policy says. So the agent opens by drawing a fraction of its line and selling it, which both funds the TOKEN side that Q1 spends and creates the outstanding issue that Q2 contracts. A central bank's asset side is bought with money it issued, and the same structure is what gives it room to move in either direction.

And the residual gap must be MEASURED rather than assumed away. Count the days the policy asked for an outright retirement and no drawn line was left, and report that number: it is the size of what an agent cannot do, in the units of the thing itself. In the reverting scenario it is not small.

8.3.5 Running the comparison

```
make nix-venv-sim-monetary-ops      # both runs, then the A/B table
```

Two runs of the same scenario and seed differing only in whether `MonetaryOpsAgent` is in the roster (`SIM_MONETARY_OPS=1`), compared by `alberta_buck/sim/compare_ops.py`. One agent, not a population: the basket has one operations desk, and a single actor keeps the result attributable to the mechanism rather than to a crowd of them meeting each other's impact.

Report the desk's terminal INVENTORY beside its P&L, always. A desk that damps the deviation by accumulating a position it could never unwind has moved the loss rather than removed it, and the deviation series alone will not say so – that is exactly the first runaway in this article, which showed a profit in seven seeds of nine while holding half the pool.

8.3.6 Units and orientation, which are the two silent killers

`bvib` is BUCK PER BASKET. TOKEN to BUCK multiplies; BUCK to TOKEN divides. Getting this backwards still runs, still trades, and reports plausible numbers – it simply sizes every operation wrongly and marks the book wrong.

`Buck.balanceOf` on a Non-Carrying account is held BUCK PLUS unused credit headroom, so before any draw it already reports `creditLimit` = K x collateral. That is the drawable amount and it is what a position should be sized against. `signedBalanceOf` is the raw balance and does NOT net the demurrage lien, so sizing a transfer off it produces "amount exceeds spendable".

Pool orientation decides which side of the current tick a directional position sits on. The pool price is `token1/token0`, so holding BUCK to sell into a BUCK rally is ABOVE the tick when BUCK is `token0` and BELOW it when BUCK is `token1`. Getting this wrong puts every position on exactly the wrong side and looks like bad luck.

8.3.7 Concentrated liquidity is not full-range liquidity

The seeding formula in `deploy.py` is for full-range positions. Reusing it for a narrow band under-sizes by the ratio of the widths – 20 to 30x at 300 to 600bp – which presents as positions placing successfully while most of the capital sits idle. With the price inside the range both legs bind:

$$a_0 = L \frac{\sqrt{B} - \sqrt{P}}{\sqrt{P}\sqrt{B}}, \quad a_1 = L(\sqrt{P} - \sqrt{A}), \quad L = \min(L_0, L_1)$$

A tick is a log price, so a 'bp' move is $\ln(1+x)/\ln(1.0001)$ ticks – 900bp is 862 ticks, not 900. Using bp directly makes a wide range 4% narrower than intended.

8.3.8 Telemetry, or the agent is invisible

Counters written into `ctr` are NOT in the vector unless `snapshot.py` copies them into the frame dict. A counter that is not copied reads 0 forever, and that has produced two separate "the agent is dead" conclusions on this branch about agents that were working. Add the counters to `snapshot.py` in the same commit as the agent, never later.

Report per-agent inventory through `arb_state()`, which the snapshotter collects into `arb2[]`. Aggregate counters hide the failure that matters here: an agent whose position is quietly becoming the market shows healthy totals right up until it cannot exit.

8.3.9 Where each bound is enforced

bound	value	enforced
per-operation	0.4% depth	trade sizing, via <code>_impact_cap</code>
position	10% depth	refuse to add when inventory exceeds it
escalation	5% depth	switch temporary -> outright
cumulative	10% supply	net (minted - burned) across the run
measurement scale	20d rung	never slower than 40d (80d inverts)

8.3.10 Verification checklist

Before trusting any result from it:

1. counters appear in the vector and are non-zero;
2. `arb_state` inventory is bounded and NOT monotonically growing;
3. outright operations fire at all – if Q2/Q4 are zero the mechanism is half inert, which is how this model first behaved;
4. supply moves in the correct direction during an attack, not merely at the end of the run;
5. a run with the agent disabled, same seed, for the baseline;
6. several seeds, because the failure case reversed sign between one seed and nine.

8.4 Order of work

1. The model in this article (done): does the mechanism work, and where does it fail?
2. An *agent* that runs the quadrants against the live chain sim (done, and see *What the agent showed* below), so the behaviour is observed against real pools and real counterparties before any contract changes.
3. `monetaryEffort()` on the director (done).
4. `monetaryOperation()` on the basket (done), with the separate liability class.

8.5 What the agent showed, including the part that did not survive

`MonetaryOpsAgent` runs the quadrants from outside the basket. Over 365 days in the reverting regime, against the same roster and seed with the agent removed:

seed	peak dev off -> ON	mean dev off -> ON	depositor P&L	treasury
default	1207 -> 821	378 -> 287	+106,984	-106,692
7	1098 -> 1154	344 -> 301	+138,237	-97,106
11	1081 -> 1246	353 -> 286	+100,458	-94,094

The first row alone reads as a 32% cut in the peak excursion. It is a single draw, and it does not replicate: on two further seeds the PEAK got WORSE, by 56bp and 165bp. What survives all three is different and narrower – the MEAN deviation falls every time, by 12 to 24%.

The distinction is the useful part. A bounded desk absorbing a fraction of depth per day can hold down the everyday wander of the common mode; it cannot stand in front of the excursion

that actually matters, and its own standing issuance adds float while it tries. Anyone quoting a peak-damping number from one seed is quoting noise, and this article did exactly that once already, in the `drift` row of the scenario table.

What also replicates, in all three seeds, is a transfer: depositors gain about \$100-140k and the treasury loses very nearly the same amount. The plausible mechanism is that holding BUCK nearer parity puts fewer redemptions into BuckBasketProRata’s deflation branch, which is the branch that converts depositors’ TOKEN to cover burns – so the treasury accrues less because it is expropriating depositors less. That is an interpretation of a consistent number, not a measured causal chain.

The agent also measured its own ceiling. `moNoBook` counted 36 days on which the policy called for an outright retirement and the agent had no drawn line left to retire against – the gap that only the contract can close.

9 The contract: BuckBasketOps

`src/basket/BuckBasketOps.sol`, with `commonMode()` / `monetaryEffort()` on `PairsRebalanceDirector`. Selected with `--basket ops`, and inert until `setOpsParams` enables it, so an ops basket with no policy installed is the baseline it is compared against.

9.1 The depositor separation is achieved by NOT writing to the payout path

This was expected to be the expensive part and turned out to be the cheapest, because the architecture already had the shape.

A depositor’s claim is $V = \theta \text{NAV}$ with $\theta = \text{redeemBuck} / \text{totalOutstandingBuck}$, allocated from `poolBuckValues()`, which reads the basket’s LP POSITION net of `treasuryLiquidity`. There is already a second, non-depositor class of liquidity, and exactly ONE line excludes it:

```
if (totalL <= c.treasuryLiquidity) continue;
depL[i] = totalL - c.treasuryLiquidity;
```

So the monetary book is held as plain TOKEN and BUCK BALANCES and never LP’d. It is then outside `depL` by construction, and not one line of the redemption allocator changes. `mintFromBasket` does not touch `totalOutstandingBuck` either – only `depositToken`, `_depositBuck` and `redeem` do – so monetary issuance cannot enter a receipt’s denominator even by accident.

The reason to prefer this over netting a monetary term out of the payout formula: θ appears four times across `_allocateSellHigh` and `_allocateSingleToken`, and a mistake in any of them is a silent transfer away from depositors rather than a bad trade. Two tests pin it – the book survives a full depositor exit, and no quadrant moves `totalOutstandingBuck`.

9.2 What only the contract can do

- **Q2 retires float the basket never issued.** `burnFromBasket` destroys BUCK bought from anyone. An agent cannot: supply is $\sum_a \max(0, \text{signedRaw}(a))$, so it removes float only while its own balance is drawn, and `Buck.burn` merely deactivates coverage.
- **Q4 is not capped by K.** `mintFromBasket` is backed by basket TOKEN rather than by insured-asset credit, so it bypasses `creditLimit` entirely.

9.3 A bound that is not politeness

`maxLegBp` caps each operation at a fraction of that pool's BUCK reserve. That is a safety property, not good manners: `poolBuckValues` runs `_enforceSlippageGuard` on every redemption, so a monetary swap large enough to push spot off TWAP would revert every depositor exit until the window caught up. The basket must not be able to brick its own redemption path.

9.4 The desk has to be capitalized, and that is the 1992 lesson

Q1 and Q2 are TOKEN-funded bids. The desk may not fund them from depositor TOKEN – a depositor is paid in TOKEN only, so converting their commodity to defend parity would charge them for the defence. Its own Q4 issuance is the other source, but that is exactly backwards for a desk whose first task is an excursion the WRONG way: in the reverting regime BUCK is never dear, so the desk never accumulates. Uncapitalized, it wanted to act on 57 of 90 days and had nothing to act with.

A currency board can only defend with reserves it built up earlier. So the desk is capitalized like one – `capitalizeMonetary()` is the founding grant, and in production the same job is done by treasury accrual, which is already outside every depositor claim. The size of that grant is a real policy variable and it is the first thing to sweep.

9.5 What the contract desk did

Two seeds, 365 days, the same roster and seed in both arms; the only difference is which shell is deployed (the monetary-ops AGENT is off in both, so this isolates the contract).

	43981 base	ops	seed 7 base	ops
peak deviation bp	1143	768	1083	914
mean deviation bp	389	289	358	322
terminal dev bp	755	620	723	-48
BUCK supply	25.34M	26.13M	24.38M	24.38M
depositor P&L	+219,966	+320,137	+194,826	+279,558
treasury BUCK	413,163	354,035	424,356	384,967
days buckK railed	1	1	1	1
Q1 / Q2 / Q3 / Q4	-	89/26/0/2	-	87/14/36/10

The peak improves in BOTH seeds, which is the line the AGENT could not hold – it made the peak worse in two seeds of three. The difference is Q2 and Q4: `burnFromBasket` retires float the basket never issued, and `mintFromBasket` is not capped by K.

9.5.1 The bound that prevented the escalation it was meant to trigger

Q2 fired ZERO times in the first pair of runs, while the inventory ceiling bound on 150 and 95 days. `_absorb` tested that ceiling at the top and reverted before Q2 was ever considered – so a desk pinned at its ceiling could never escalate, though being pinned there IS the signal that the move is real. The bound meant to trigger escalation was preventing it: the first runaway of this article, one layer further out.

Q2 now burns INVENTORY first, which needs no TOKEN (by the time persistence is established the desk has usually spent its reserves absorbing) and relieves the ceiling. On seed 43981 that took the peak from -138bp to -375bp against baseline, halved the supply increase, cut the ceiling hits

from 150 to 32, and moved the monetary book from +73k issued to -2.05M retired. On seed 7 the net supply change went from +870k to -9k: the retirement exactly cancelled the issuance.

9.5.2 Reducing supply and profiting from it are the same budget spent twice

seed	desk P&L	float retired	retired per BUCK spent
43981	-1.99M	2.05M	1.03
7	-0.49M	0.59M	1.22

The desk LOSES on its own book and retires slightly MORE float than the book it spent, because it is buying BUCK at a discount and destroying it. That is not a defect, it is the trade: Q1 and Q3 are the extraction path – buy the dump, sell it back, keep the spread – and Q2 forgoes that recovery profit to buy a permanent supply reduction instead. The gain accrues to every BUCK holder rather than to the desk, and it shows up here as depositors ending \$85-100k better off in both seeds.

So the answer to "can we profit from reducing BUCK supply AND reduce BUCK supply" is: not with the same operation. A desk tuned to extract holds and sells; a desk tuned to stabilize burns. What the persistence test buys is the ability to do the first by default and the second only when the excursion proves real.

9.5.3 K kept forcing

buckK railed for exactly one day in all four runs, and the desk retired 2.05M and 0.59M of float without changing that. At this size the fast operations are not damping K's error into silence, which is the precondition for the desk's position being a bet worth placing at all. Whether it survives a larger desk is exactly what `SIM_OPS_CAPITAL_USD` is for, and it is the open question rather than a settled one.

9.6 Why the limits exist, and what they are actually for

The bounds are not primarily risk limits. They exist to stop the desk substituting a fast fix for the slow, structural withdrawal of BUCK that K performs through `creditLimit` – so they are the knob that sets where the two mandates OVERLAP, and they are meant to be swept (`SIM_OPS_POSITION_BP / SIM_OPS_OUTRIGHT_BP / SIM_OPS_CAPITAL_USD`).

This reframes what counts as success. The desk's position is a BET that K eventually forces the issue: it buys BUCK the market is dumping and holds it while K tightens credit behind it. Interim mark-to-market is therefore the wrong scorecard, and a desk that loses in the short term is fine provided the long-term forcing arrives.

What would invalidate the bet is subtler and is the thing to watch for. The desk damps the excursion by absorbing it, and `basketValueInBuck` is read from the very pools it absorbs into – so a SUCCESSFUL operation shrinks the error K sees. If K then stops tightening, the forcing never arrives and the desk is left holding inventory with nothing behind it. That is the system-level form of the first runaway in this article: the operator suppressing its own alarm, one loop further out.

`monetaryDeviationOffset()` exposes exactly that quantity – what the desk's own inventory has taken out of the deviation K measures. It is EXPOSED rather than acted on, because the right response is a controller-side choice (feed K the undamped error, or feed the achieved action back to the integrator as anti-windup) and it should be measured before it is picked. In a 90-day run the desk held 1.25M BUCK of float while buckK still fell from 0.745 to 0.584, so the forcing was not

being suppressed at that size. Whether that survives a larger desk is the open question, and it is the reason the bounds are a sweep rather than a constant.

10 Limitations

- The model abstracts every pool to one price. Real operations act on one TOKEN/BUCK pool at a time and must choose which, which is a routing problem the differential engine already solves and this model does not exercise.
- Attack sizing is a single shape: 6M over 20 days, held, then unwound. A patient attacker who bleeds size in over months is a different problem and is not tested here.
- The reversion timescale is assumed, not fitted. The basket’s real reversion rate should come from the same OU fit that the rebalance article proposes for its deadbands.
- Nothing here models the interaction between monetary operations and the commodity rebalancer competing for the same pool depth. They share an actuator, and that coupling is unexamined.
- The 1:K position sizing that a real range order needs – placing an exact mix over the current price – is solved in `BuckPoolInvestorAgent` but not reflected in this model, which trades a single abstract price.
- **The parameterization is not converged.** The bounds above are the ones that stopped the runaways this model produced, at the sizes that happened to work; they are not fitted, and no claim is made that they are optimal or even good. The end-of-run supply change also reflects the whole path rather than the attack alone – an inflation attack contracts supply while it runs and expands it again as the attacker unwinds, so the terminal number is not a clean read on the response.